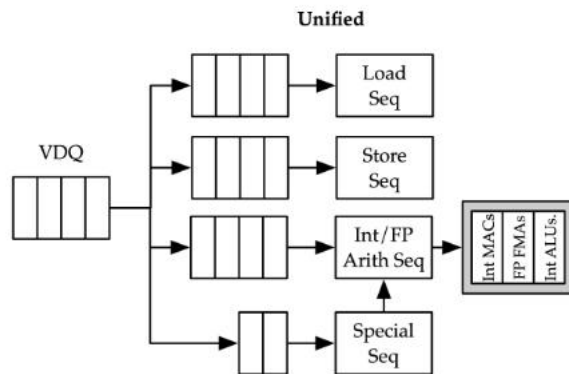


Survey on Microarchitecture (Saturn, XuanTie910 and K3)

1. Vector Backend

1.1 Saturn:

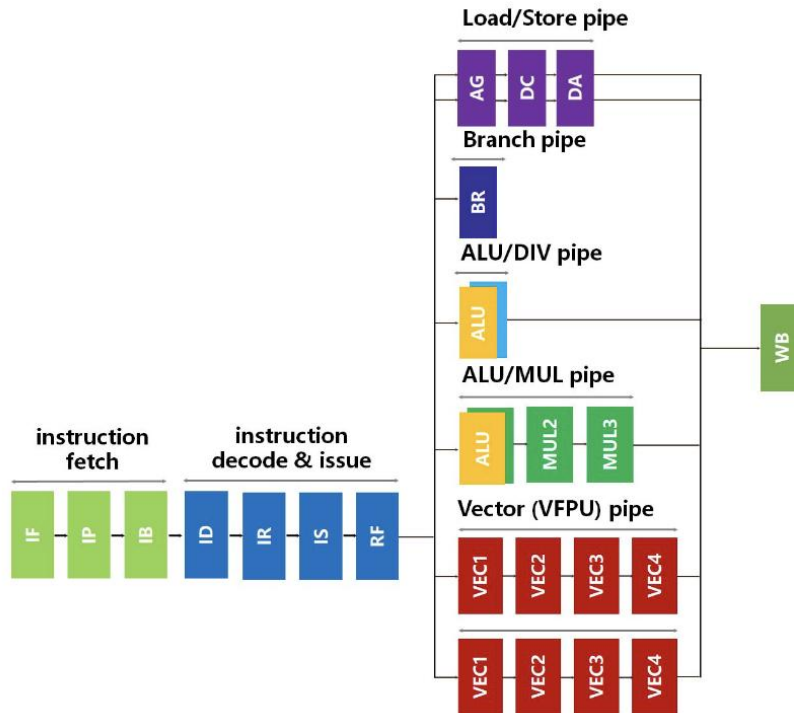


Vector Frontend (VFU) is integrated into the scalar core. It will send vector instructions together with vector CSRs (vtype, vl, lmul, vstart) information to Vector Dispatch Queue (VDQ) in Vector Datapath (VU).

VDQ will further dispatch instructions to different Vector Issue Queues (VIQs), based on the type of instructions. Each VIQ issues instructions in-order into its corresponding instruction sequencer. Execution across the VIQs may be out-of-order, as the VIQs may naturally “slip” against each other. (I don’t think OoO on micro-ops granularity)

The sequencers decompose the instructions to a sequence of micro-op that execute down the functional unit datapaths, one operation per cycle. Sequencers advertise current instructions information(which DLEN bits of element group is executing and age tag) to other sequencers. Each sequencer keeps track of instructions in the issue queues, the instructions currently in progress within the sequencers, and any operations which have not yet completed execution in the functional unit datapaths. Based on these information, sequencer decide whether to send micro-op to execute or stall the datapath.

1.2 XuanTie-910



Vector registers are renamed in IR stage. Vector instructions are OoO scheduled in IS stage, together with scalar instructions (Age-Vector based scheduling algorithm is used).

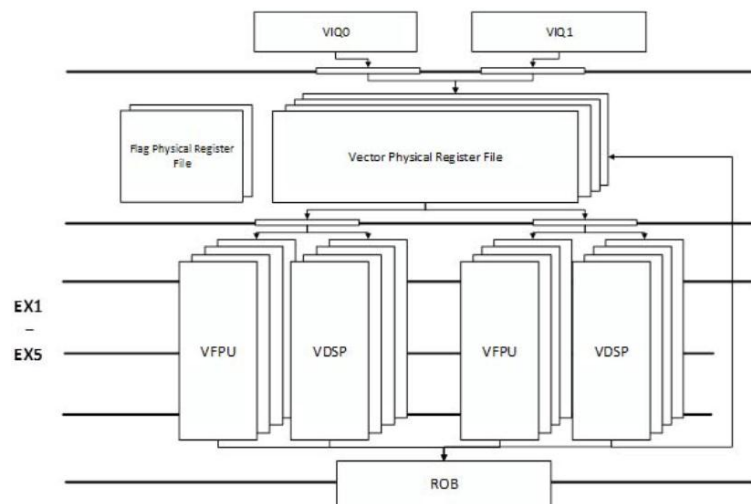


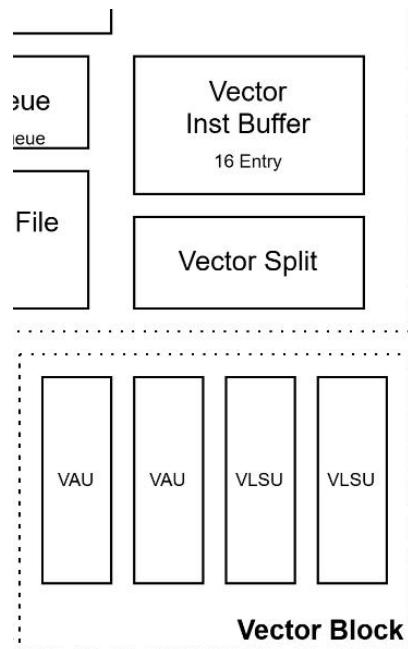
Fig. 14: The pipelined vector operation architecture.

Vector pipeline could consist of multiple slices (recommend 2 slices). A slice (lane) consists of a multi-port (Doesn't mention specific port numbers) 64 bit vector physical register file and two OoO vector floating-point and integer execution pipelines. Each slice issues two vector instructions, they could be executed OoO.

Recommended VLEN = 128, DLEN = 64.

$2 \text{ slices} * 2 \text{ VXU/slice} * 64 \text{ bit DLEN} = 256 \text{ bits data / cycle}$

1.3 K3 X100 Core



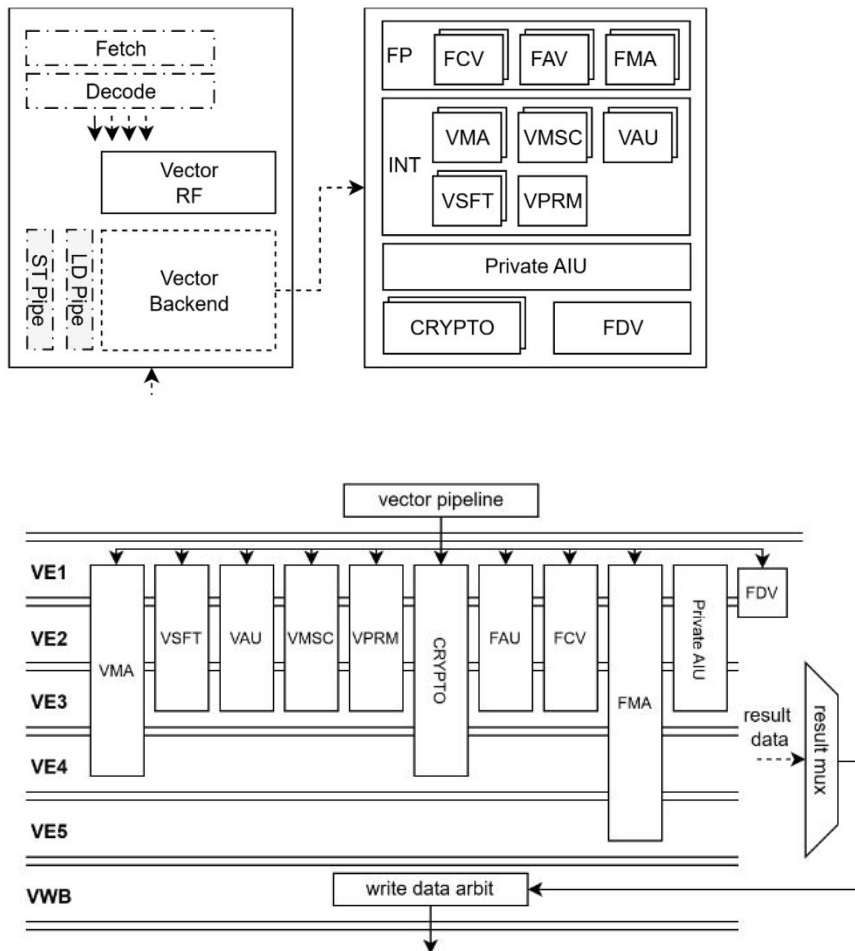
After decoding and renaming, vector instructions are sent to Vector Instruction Buffer for further execution.

VLEN = 256, ELEN = 64 The vector unit supports the **concurrent issue of up to two memory instructions and two computational instructions**. Each issue port can handle an instruction micro-op in 128-bit granularity per cycle (DLEN = 128), which means an instruction with LMUL=1 is issued as two 128-bit micro-ops.

Seems like load store instructions and arithmetic instructions could be issued at the same time -> Out of Order.

No more information about pipeline stage.

1.4 K3 A100 Core



VLEN = 1024, DLEN depends.

Domain	Unit / Condition	Resource Provisioning (channels $\times$ width)	Result Stage
INT	VAU (add/sub/compare/minmax)	2×256	VE3
INT	VSFT (shift/rotate/sat-shift)	2×256	VE3
INT	VMA (SEW=8/16)	2×256	VE3
INT	VMA (SEW=32/64)	2×128	VE4
INT	VMSC (mov/logic)	2×256	VE3
INT	VMSC (others, e.g., clz/cpop)	2×128	VE3
INT	VPRM (slide/gather/compress/transpose, etc.)	2×256	Implementation-dependent
FP	FAU (SEW=16/32)	2×256	VE3
FP	FAU (SEW=64)	2×128	VE3
FP	FMA (SEW=16/32)	2×256	VE4
FP	FMA (SEW=64)	2×128	VE5
FP	FCV (SEW=16/32)	2×256	VE3
FP	FCV (SEW=64)	2×128	VE3
CRYPTO	128-bit group (AES/GHASH / subset of SHA-2)	2×256	VE3
CRYPTO	256-bit group (SM3/SM4 / subset of SHA-2)	2×128	VE4
FDV	div/rem/sqrt (SRT iteration; shared int/FP resources)	1×128	At iteration completion

Table 2: Execution resource of A100 Vector Execution Unit.

Paper doesn't mention the pipeline stage details, and execution order details. But some instructions could be dual-issued .

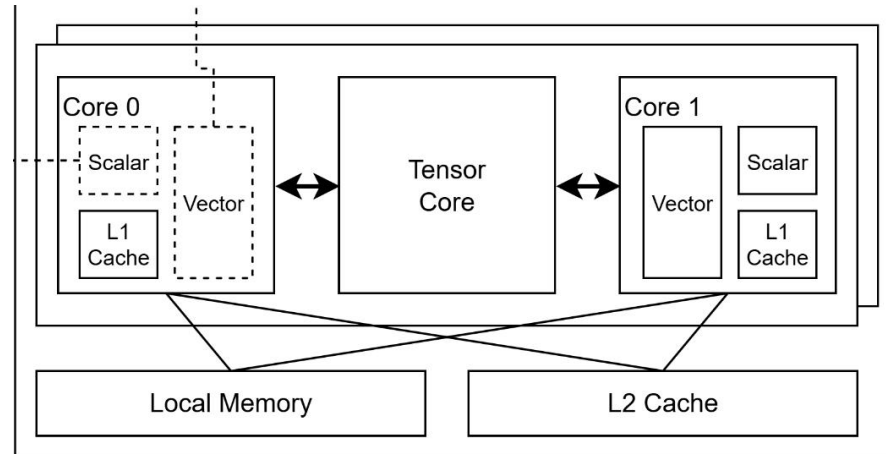
throughput-critical operators (add/sub, compare, shift, logic, and narrow-width multiply-accumulate) are preferentially mapped to wider execution resources capable of dual-issue,

and are scheduled to produce results as early as possible in VE3 (or VE4).

So I guess micro-ops across different execution units are OoO.

2. Memory sub-system (TCM)

K3 A100 Core use on-chip local memory.



每个 A100 集群包含 "two local-memory banks"。两个 A100 Cores 共享一个 Local memory。它们同时与 Tensor Core 共享，用于提高张量计算资源利用率和数据重用效率。

"The design implements software-managed local memory and decoupled access-execute operation"

local memory 是由软件管理的。

此外。"With RVV configured at VLEN=1024 and 32 architectural vector registers, the programming model can be viewed as providing an effective 4 KB ultra-fast local scratchpad." K3 将大容量向量寄存器文件本身视为一种 scratchpad memory, 提供 4KB 的超快本地存储空间。